

Aula 009, ligacao estatica x dinamica. Enunciado no aula009.pdf.

1 Java

```
class Animal {  
    String som() { return "..."; }  
}  
class Cachorro extends Animal {  
    String som() { return "au"; }  
}  
Animal x = new Cachorro();  
System.out.println(x.som());
```

Saida: au

Decisao: execucao. A variavel e Animal mas o objeto de verdade e Cachorro. Metodo de instancia no Java e polimorfico, quem manda e o tipo real na hora de rodar, nao o tipo da referencia.

2 C++

```
struct A {  
    void f() { cout << "A"; }  
};  
struct B : A {  
    void f() { cout << "B"; }  
};  
B b; A *p = &b;  
p->f();
```

Saida: A

Decisao: compilacao. Sem virtual o compilador olha o tipo do ponteiro (A*) e ja fixa a chamada em A::f. Mesmo apontando pra um B, nao troca na execucao.

3 Java

```
class A {  
    String nome = "A";  
    String getNome() { return nome; }  
}  
class B extends A {  
    String nome = "B";  
    String getNome() { return nome; }  
}  
A x = new B();  
System.out.println(x.nome + " " + x.getNome());
```

Saida: A B

Decisao: compilacao e execucao, cada pedaco de um jeito. Campo nome nao e polimorfico, x.nome usa o tipo da referencia A, da A na compilacao. Ja getNome() e metodo, chama o de B na execucao e devolve o nome de B.

4 Python

```
class Contador:
    total = 0
    def __init__(self):
        Contador.total += 1
        self.id = Contador.total

a = Contador(); b = Contador()
print(a.id, b.id, a.total)
```

Saida: 1 2 2

Decisao: execucao. id e por instancia, total e atributo de classe compartilhado. So da pra saber os valores depois de criar a e b e ir incrementando.

5 Java

```
class A {
    static String quem() { return "A"; }
}
class B extends A {
    static String quem() { return "B"; }
}
A x = new B();
System.out.println(x.quem());
```

Saida: A

Decisao: compilacao. Metodo estatico nao participa de polimorfismo de instancia. x e declarado como A, entao quem() resolve como A::quem ainda na compilacao, mesmo o new sendo B.

6 Go

```
type Animal struct{}
func (Animal) Som() string { return "..."}
func (a Animal) Falar() string {
    return "faz " + a.Som()
}
type Cao struct{ Animal }
func (Cao) Som() string { return "au"}

fmt.Println(Cao{}.Falar(), Cao{}.Som())
```

Saida: faz ... au

Decisao: compilacao. Falar veio embutido de Animal e o receiver dentro dele e Animal, entao a.Som() e o Som de Animal (...), nao o override do Cao. Cao{}.Som() chama direto o metodo do Cao e ai sim da au. Go nao faz dispatch dinamico igual Java nesse caso.